

Comprendre ton jeu du pendu

Guide pas a pas pour debuter en Python

Ce document explique les fonctionnements presents dans ton dossier **hangman** avec des mots simples. Chaque partie contient l'enonce, l'idee, un exemple de ton exercice et une petite tache a refaire seul.

1. Avant de commencer

Un programme est une suite d'instructions. Python lit ces instructions de haut en bas. Dans un jeu, certaines instructions sont repetees dans une boucle et d'autres sont rangees dans des fonctions.

Les mots importants

- **Variable** : une boite qui garde une valeur. Exemple : `penalties = 0`.
- **Fonction** : un petit bloc de code reutilisable. Elle commence par `def`.
- **Condition** : une question qui decide quoi faire. Elle utilise `if`, `elif` et `else`.
- **Boucle** : du code repete. Ici, on utilise `while` pour continuer la partie.
- **Evenement** : une action du joueur, par exemple un clic ou la fermeture de la fenetre.

Le plan general du pendu

1. Choisir un mot secret
2. Afficher des tirets a la place des lettres
3. Demander une lettre au joueur
4. Verifier si la lettre est dans le mot
5. Ajouter une erreur si elle est fausse
6. Arrêter si le mot est trouve ou si la limite d'erreurs est atteinte

2. Exercice console : choisir et afficher un mot

Enonce de l'exercice : lire un fichier contenant un mot par ligne, choisir un mot au hasard et afficher les lettres trouvees. Les lettres inconnues doivent etre remplacees par un tiret bas.

Fonction `choose_word`

```
def choose_word(arguments):
    content = arguments.word_list.read_text(encoding="utf-8")
    words = []

    for line in content.splitlines():
        word = line.strip().upper()
        if word.isalpha():
            words.append(word)

    return random.choice(words)
```

Explication : `read_text` lit le fichier. `splitlines()` separe le texte ligne par ligne. `strip()` enleve les espaces et le retour a la ligne. `upper()` transforme le mot en majuscules. Enfin, `random.choice` choisit un element au hasard.

Exemple : si le fichier contient `chat`, `python` et `bateau`, le programme peut choisir `PYTHON`. Le choix change d'une partie a l'autre.

Fonction `display_word`

```
def display_word():
    result = ""
```

```

for letter in word:
    if letter in found_letters:
        result += letter + " "
    else:
        result += "_ "

return result

```

Ici, `word` est le mot secret et `found_letters` contient les lettres déjà trouvées. `return` renvoie le résultat à l'endroit où la fonction a été appelée.

Exemple corrigé : mot = CHAT et lettres trouvées = {A, T}. L'affichage devient _ _ A T.

La boucle de jeu

```

while penalties < MAX_PENALTIES:
    guess = input("$> ").strip().upper()

    if len(guess) == 1:
        if guess in word:
            found_letters.add(guess)
        else:
            penalties += 1

```

La boucle continue tant que le nombre de pénalités est inférieur au maximum. `input` attend la réponse du joueur. `len(guess)` donne le nombre de caractères. Une lettre correcte est ajoutée dans l'ensemble `found_letters`; une mauvaise lettre augmente `penalties`.

A faire seul

- Change le nombre maximum de pénalités et observe le résultat.
- Ajoute un message quand le joueur propose une lettre déjà trouvée.
- Teste le mot CHAT avec les lettres A puis X.

3. Exercice console complet : options et score

Enonce de l'exercice : rendre le jeu configurable avec un fichier de mots obligatoire, un nombre maximum de penalites, une longueur de mot et un theme. Gerer les erreurs de fichier et enregistrer le meilleur score.

Lire les arguments

```
parser.add_argument("word_list", type=Path,
                    help="text file containing one word per line")
parser.add_argument("--penalties", type=int, default=12)
parser.add_argument("--length", type=int)
parser.add_argument("--theme", default="general")
```

Un argument est une information donnee au lancement du programme. Exemple : `python challenge.py my_wordlist.txt --penalties 6 --length 5 --theme animaux`. Le fichier est obligatoire; les autres options sont facultatives car elles ont une valeur par default.

Verifier les erreurs

```
if arguments.penalties < 1:
    parser.error("--penalties must be at least 1")

if arguments.length is not None and arguments.length < 1:
    parser.error("--length must be at least 1")
```

La condition empeche des valeurs impossibles. `is not None` signifie : une longueur a ete donnee. Une bonne verification evite que le programme continue avec une mauvaise valeur.

Exemple : avec `--penalties 0`, le programme affiche une erreur car une partie doit permettre au moins une penalite.

Les lettres et le mot complet

```
if len(guess) == 1:
    if guess in word:
        found_letters.add(guess)
    else:
        penalties += 1

elif len(guess) > 1:
    if guess == word:
        print("correct guess")
        break
    penalties = min(penalties + 5, MAX_PENALTIES)
```

Une proposition d'un seul caractere est traitee comme une lettre. Une proposition plus longue est traitee comme un mot entier. Un mauvais mot coute ici 5 penalites, mais `min` empeche de depasser la limite.

Le meilleur score

```
def get_high_score():
    # Lire highscore.txt
    # Pour chaque ligne : nombre;date
    # Garder le plus petit nombre
    return best_attempts, best_date

def save_high_score(attempts):
    with HIGH_SCORE_FILE.open("a", encoding="utf-8") as file:
        file.write(f"{attempts};{today}\n")
```

Le fichier `highscore.txt` garde une ligne comme `4;2026-09-18`. Le plus petit nombre de tentatives est le meilleur score. Le mode `a` signifie ajouter a la fin du fichier.

A faire seul

- Lance le jeu avec `--theme espace` et explique le message affiche.
- Ajoute un refus pour les caracteres speciaux comme `!`.

- Ajoute une ligne de score et vérifie qu'elle est relue au prochain lancement.

4. Exercice Pygame : la version graphique

Enonce de l'exercice : creer une fenetre Pygame, dessiner le pendu, afficher un clavier de lettres, permettre au joueur de cliquer, afficher la victoire ou la defaite et proposer un bouton Rejouer.

Initialiser Pygame et la fenetre

```
import pygame
import random
from pathlib import Path

pygame.init()
WIDTH = 800
HEIGHT = 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Hangman")
```

`pygame.init()` prepare Pygame. `screen` represente la fenetre. Sa taille est de 800 pixels de large et 600 pixels de haut.

Dessiner une forme

```
pygame.draw.rect(screen, (90, 180, 80), (0, 500, WIDTH, 100))
pygame.draw.line(screen, wood_color, (600, 500), (600, 100), 15)
pygame.draw.circle(screen, skin_color, (700, 180), 30)
```

Chaque couleur est un tuple (rouge, vert, bleu). Un rectangle represente le sol, une ligne represente la potence et un cercle represente la tete.

Faire apparaitre le personnage progressivement

```
if wrong_guesses >= 1:
    pygame.draw.circle(screen, skin_color, (700, 180), 30)

if wrong_guesses >= 2:
    pygame.draw.line(screen, clothes_color, (700, 210), (700, 300), 8)
```

La valeur `wrong_guesses` compte les mauvaises lettres. A 1 erreur, on dessine la tete; a 2 erreurs, le corps; puis les bras et les jambes. C'est une condition qui controle le dessin.

Detecter un clic sur une touche

```
rect = pygame.Rect(x, y, KEY_WIDTH, KEY_HEIGHT)

if rect.collidepoint(position):
    if letter not in used_letters:
        used_letters.append(letter)
        if letter not in secret_word:
            wrong_guesses += 1
```

`pygame.Rect` decrit la zone d'un bouton. `collidepoint` repond vrai si la souris est dans cette zone. La lettre est ajoutee une seule fois dans `used_letters`.

Connaitre l'etat de la partie

```
def check_game_status():
    if wrong_guesses >= 6:
        return "lose"

    for letter in secret_word:
        if letter not in used_letters:
            return "playing"

    return "win"
```

La fonction renvoie un mot simple : `playing`, `win` ou `lose`. Si six erreurs sont atteintes, la partie est perdue. Sinon, on cherche encore une lettre non trouvee. S'il n'en reste plus, le joueur a gagne.

5. La boucle Pygame expliquée

Recevoir les événements

```
running = True

while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

        if event.type == pygame.MOUSEBUTTONDOWN:
            if game_status == "playing":
                check_keyboard_click(event.pos)
            else:
                check_restart_click(event.pos)
```

La boucle tourne environ 60 fois par seconde. `pygame.event.get()` récupère les actions en attente. Si le joueur clique sur la croix, `running` devient `False`, donc la boucle se termine normalement. Si le joueur clique, le programme choisit entre le clavier et le bouton Rejouer.

Redessiner l'écran

```
draw_background()
draw_gallows()
draw_character()
draw_keyboard()
draw_word()
draw_game_status(game_status)

if game_status != "playing":
    draw_restart_button()

pygame.display.flip()
clock.tick(60)
```

A chaque tour, on redessine toute la scène. `pygame.display.flip()` rend le nouveau dessin visible. `clock.tick(60)` limite la vitesse à 60 images par seconde.

Rejouer une partie

```
def restart_game():
    global used_letters, wrong_guesses
    global game_status, secret_word

    used_letters = []
    wrong_guesses = 0
    game_status = "playing"
    secret_word = choose_word()
```

La fonction remet toutes les valeurs dans leur état de départ. `global` indique que la fonction modifie les variables créées en dehors d'elle.

Pourquoi le jeu se fermait après une défaite ?

```
word_rect = word_text.get_rect(center=(400, 440))
screen.blit(word_text, word_rect)
```

La variable doit garder exactement le même nom. Avant la correction, le code utilisait `word_rect` alors que la variable s'appelait `word_rect`. Python ne trouvait pas `word_rect` et arrêtait le programme avec une erreur `NameError`.

Règle débutant : quand une fenêtre se ferme toute seule, lance le programme depuis le terminal. Le message d'erreur indique souvent la ligne qui a provoqué l'arrêt.

6. Corrections et points a verifier

Correction 1 : chemin du fichier

Dans la version Pygame, `Path("my_wordlist.txt")` cherche le fichier dans le dossier depuis lequel tu lances Python. Lance donc le programme depuis le dossier `hangman`, ou utilise un chemin construit a partir du fichier Python.

```
from pathlib import Path

BASE_DIR = Path(__file__).parent
WORD_LIST_FILE = BASE_DIR / "my_wordlist.txt"
```

`__file__` represente le fichier Python actuel. `parent` represente son dossier. Cette version est plus solide si tu lances le programme depuis un autre endroit.

Correction 2 : accents dans la liste

Dans `challenge.py`, le test `word.isascii()` refuse les lettres accentuees. Le mot électrique sera donc ignore. Pour un jeu limite aux lettres A-Z, c'est normal. Pour accepter le francais, il faut retirer ce test et choisir une regle claire pour les accents.

Correction 3 : fichiers vides

`highscore.py` et `task 3.4.py` sont actuellement vides. Ce n'est pas une erreur : ce sont des fichiers prevus pour des exercices ou des extensions a completer.

Methode pour travailler seul

- Lis l'enonce et souligne les donnees a memoriser : mot, lettres, erreurs, score.
- Ecris d'abord une petite fonction qui fait une seule chose.
- Teste avec un exemple tres petit : mot = CHAT.
- Observe le message d'erreur avant de modifier le code.
- Ajoute une seule modification, puis relance le programme.

7. Exercices d'entrainement

Exercice A : ecris une fonction `mask_word(word, found_letters)` qui transforme PYTHON en _ Y _ H _ N quand les lettres trouvees sont Y, H, N.

Correction : parcourir chaque lettre avec une boucle. Ajouter la lettre si elle est dans `found_letters`, sinon ajouter _.

```
def mask_word(word, found_letters):
    result = []
    for letter in word:
        if letter in found_letters:
            result.append(letter)
        else:
            result.append("_")
    return " ".join(result)
```

Exercice B : ecris une fonction `is_won(word, found_letters)` qui renvoie True seulement si toutes les lettres du mot ont ete trouvees.

Correction : utiliser `all`. Cette fonction renvoie vrai seulement si toutes les conditions sont vraies.

```
def is_won(word, found_letters):
    return all(letter in found_letters for letter in word)
```

Exercice C : ajoute un bouton Rejouer qui remet les erreurs a zero, vide les lettres utilisees et choisit un nouveau mot. La correction est la fonction `restart_game` de `full_game.py`.

Tu peux maintenant cacher les corrections, refaire les trois exercices, puis comparer ton resultat avec ce guide.